

1. Factorial of a Number

Algorithm / Pseudocode

ALGORITHM Factorial(N)

Step 1: Start

Step 2: Read N

Step 3: If N = 0, then

Factorial $\leftarrow$ 1

Go to Step 6

Step 4: Calculate

Factorial $\leftarrow$ N $\times$ Factorial(N - 1)

Step 5: Return Factorial

Step 6: Display Factorial

Step 7: Stop

Example

N = 5

$$5 \times 4 \times 3 \times 2 \times 1$$

$$= 120$$

AI concept: Recursion

2. Name and Date of Birth Database

Algorithm / Pseudocode

ALGORITHM Name_DOB_Database

Step 1: Start

Step 2: Create a database containing:

Name

Day

Month

Year

Step 3: Store person details in the database.

Step 4: Read the required Name.

Step 5: Search the database for the given Name.

Step 6: If Name is found, then

Retrieve Day, Month and Year.

Step 7: Display Name and Date of Birth.

Step 8: If Name is not found, display "Person not found".

Step 9: Stop

Example

Input:

john

Search database

Output:

Name: john

Date of Birth: 15/5/2000

AI concept: Knowledge representation / Database facts / Logical querying

The screenshot shows a Prolog program in a text editor and its execution output in a separate window. The program is a recursive function to calculate the sum of natural numbers from 1 to N.

```
1 % Sum of Natural Numbers
2
3 sumto(1, 1).
4
5 sumto(N, S) :-
6     N > 1,
7     N1 is N - 1,
8     sumto(N1, S1),
9     S is S1 + N.
10
11 :- initialization(main).
12
13 main :-
14     sumto(10, S),
15     write('Sum of natural numbers = '),
16     write(S),
17     nl.
```

The output window shows the execution of the `member` predicate, which is used to check if an element is a member of a list. The output shows the results of the query `member(X, [cat, mouse]).` for different values of X.

```
member(X, [cat, mouse]).
X = cat
X = mouse
member(X, [cat, mouse]).
X = cat
X = mouse
member(X, [cat, mouse]).
X = cat
Next 10 100 1,000 Stop
member(X, [cat, mouse]).
Sum of natural numbers = 55
X = cat
Next 10 100 1,000 Stop
?- member(X, [cat, mouse]).
```

The screenshot shows the SWISH Prolog environment. The top menu bar includes File, Edit, Examples, and Help. Below the menu bar is a toolbar with various icons. The main area displays a Prolog program for calculating factorials.

```
factorial_func(0, 1) :- !.
factorial_func(N, Result) :-
    N > 0,
    N1 is N - 1,
    factorial_func(N1, SubResult),
    Result is N * SubResult.

debug_example :-
    Num = 5,
    factorial_func(Num, Result),
    write('Factorial of '), write(Num), write(' is '), write(Result).

main :- debug_example.
:- initialization(main).
```

A syntax error is shown at the bottom of the editor:

```
Syntax error: Operator expected (in_http_request)
```

3. Student–Teacher–Subject Code

Algorithm / Pseudocode

ALGORITHM Student_Teacher_Subject

Step 1: Start

Step 2: Create a database containing:

Student

Subject

Teacher

Subject Code

Step 3: Store student-subject relationships.

Step 4: Store teacher-subject-code relationships.

Step 5: Read the Student name.

Step 6: Find the Subject associated with the Student.

Step 7: Find the Teacher who teaches that Subject.

Step 8: Find the Subject Code of that Subject.

Step 9: Display:

Student

Teacher

Subject Code

Step 10: Stop

Example

Input:

john

john → mathematics

mathematics → teacher1

mathematics → m101

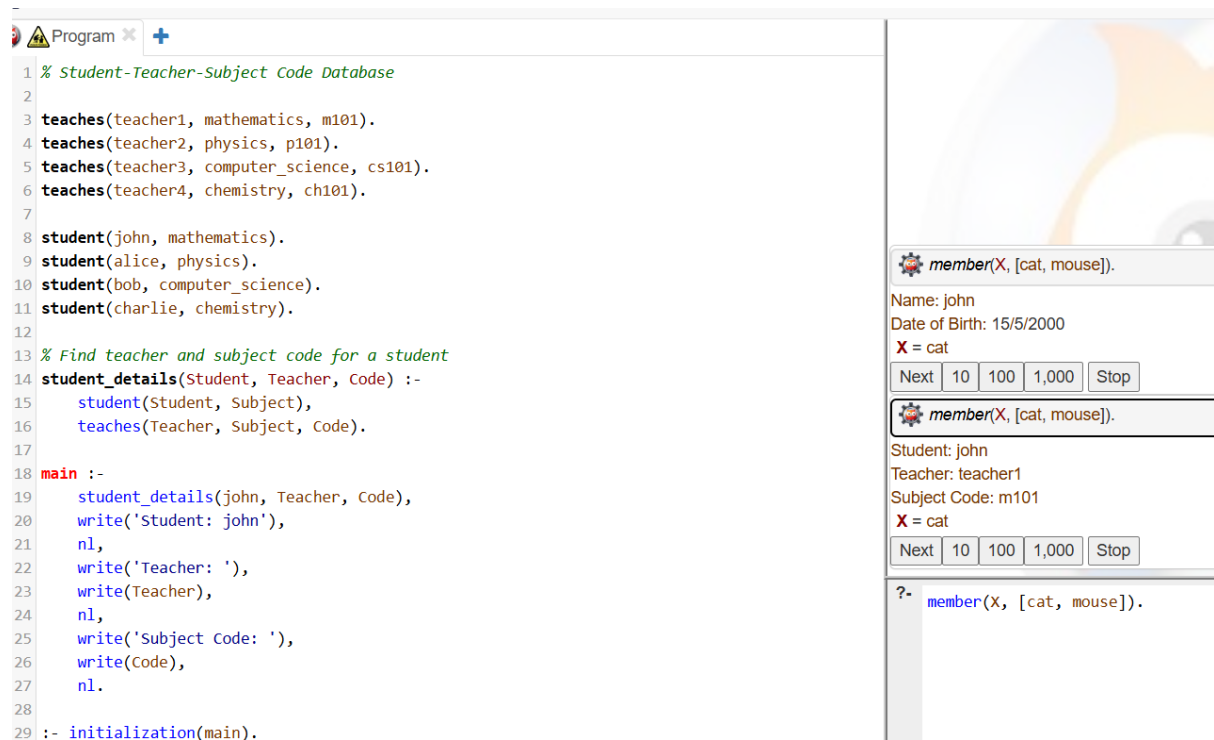
Output:

Student: john

Teacher: teacher1

Subject Code: m101

AI concept: Knowledge representation + Rule-based reasoning



```
1 % Student-Teacher-Subject Code Database
2
3 teaches(teacher1, mathematics, m101).
4 teaches(teacher2, physics, p101).
5 teaches(teacher3, computer_science, cs101).
6 teaches(teacher4, chemistry, ch101).
7
8 student(john, mathematics).
9 student(alice, physics).
10 student(bob, computer_science).
11 student(charlie, chemistry).
12
13 % Find teacher and subject code for a student
14 student_details(Student, Teacher, Code) :-
15     student(Student, Subject),
16     teaches(Teacher, Subject, Code).
17
18 main :-
19     student_details(john, Teacher, Code),
20     write('Student: john'),
21     nl,
22     write('Teacher: '),
23     write(Teacher),
24     nl,
25     write('Subject Code: '),
26     write(Code),
27     nl.
28
29 :- initialization(main).
```

member(X, [cat, mouse]).

Name: john
Date of Birth: 15/5/2000
X = cat
Next 10 100 1,000 Stop

member(X, [cat, mouse]).

Student: john
Teacher: teacher1
Subject Code: m101
X = cat
Next 10 100 1,000 Stop

?- member(X, [cat, mouse]).

4. Planets Database

Algorithm / Pseudocode

ALGORITHM Planet_Database

Step 1: Start

Step 2: Create a database containing:

Planet

Planet Type

Step 3: Store planet information.

Step 4: Read the required Planet.

Step 5: Search the database for the Planet.

Step 6: If Planet is found, then

Retrieve its Type.

Step 7: Display the Planet and its Type.

Step 8: Stop

Example

Input:

earth

Search database

Output:

Earth is a terrestrial planet.

AI concept: Knowledge representation / Facts / Logical inference

```
1 % Planets Database
2
3 planet(mercury, terrestrial).
4 planet(venus, terrestrial).
5 planet(earth, terrestrial).
6 planet(mars, terrestrial).
7 planet(jupiter, gas_giant).
8 planet(saturn, gas_giant).
9 planet(uranus, ice_giant).
10 planet(neptune, ice_giant).
11
12 % Display planet information
13
14 main :-
15     write('Planets Database'),
16     nl,
17     nl,
18     write('Earth is a '),
19     planet(earth, Type1),
20     write(Type1),
21     nl,
22     write('Jupiter is a '),
23     planet(jupiter, Type2),
24     write(Type2),
25     nl.
```

Name: john				
Date of Birth: 15/5/2000				
X = cat				
Next	10	100	1,000	Stop
 member(X, [cat, mouse]).				
Student: john				
Teacher: teacher1				
Subject Code: m101				
X = cat				
Next	10	100	1,000	Stop
 member(X, [cat, mouse]).				
Planets Database				
Earth is a terrestrial				
Jupiter is a gas_giant				
Neptune is a ice_giant				
X = cat				
Next	10	100	1,000	Stop
?- member(X, [cat, mouse]).				

5. Towers of Hanoi

Algorithm / Pseudocode

ALGORITHM Towers_Of_Hanoi(N, Source, Destination, Auxiliary)

Step 1: Start

Step 2: If $N = 1$, then

Move disk from Source to Destination.

Return.

Step 3: Move $N-1$ disks from

Source $\rightarrow$ Auxiliary

using Destination.

Step 4: Move the largest disk from

Source $\rightarrow$ Destination.

Step 5: Move $N-1$ disks from

Auxiliary $\rightarrow$ Destination

using Source.

Step 6: Stop

Example for 3 disks

Source = Left

Destination = Right

Auxiliary = Middle

Moves:

1. Left $\rightarrow$ Right

2. Left $\rightarrow$ Middle

3. Right → Middle

4. Left → Right

5. Middle → Left

6. Middle → Right

7. Left → Right

AI concept: Recursion + Problem solving

Easy way to remember

FACTORIAL

↓

RECURSION

NAME-DOB

↓

DATABASE → SEARCH → RESULT

STUDENT

↓

STUDENT → SUBJECT → TEACHER → CODE

PLANETS

↓

PLANET → TYPE

HANOI

↓

N-1 → MOVE → N-1

.

Program

```
1 % Towers of Hanoi
2
3 hanoi(1, Source, Destination, _) :-
4     write('Move disk from '),
5     write(Source),
6     write(' to '),
7     write(Destination),
8     nl.
9
10 hanoi(N, Source, Destination, Auxiliary) :-
11     N > 1,
12     N1 is N - 1,
13     hanoi(N1, Source, Auxiliary, Destination),
14     write('Move disk from '),
15     write(Source),
16     write(' to '),
17     write(Destination),
18     nl,
19     hanoi(N1, Auxiliary, Destination, Source).
```

Next101001,000Stop

member(X,[cat,mouse]).

Planets Database

Earth is a terrestrial
Jupiter is a gas_giant
Neptune is a ice_giant
X = cat

Next101001,000Stop

member(X,[cat,mouse]).

Towers of Hanoi for 3 disks:
Move disk from left to right
Move disk from left to middle
Move disk from right to middle
Move disk from left to right
Move disk from middle to left
Move disk from middle to right
Move disk from left to right